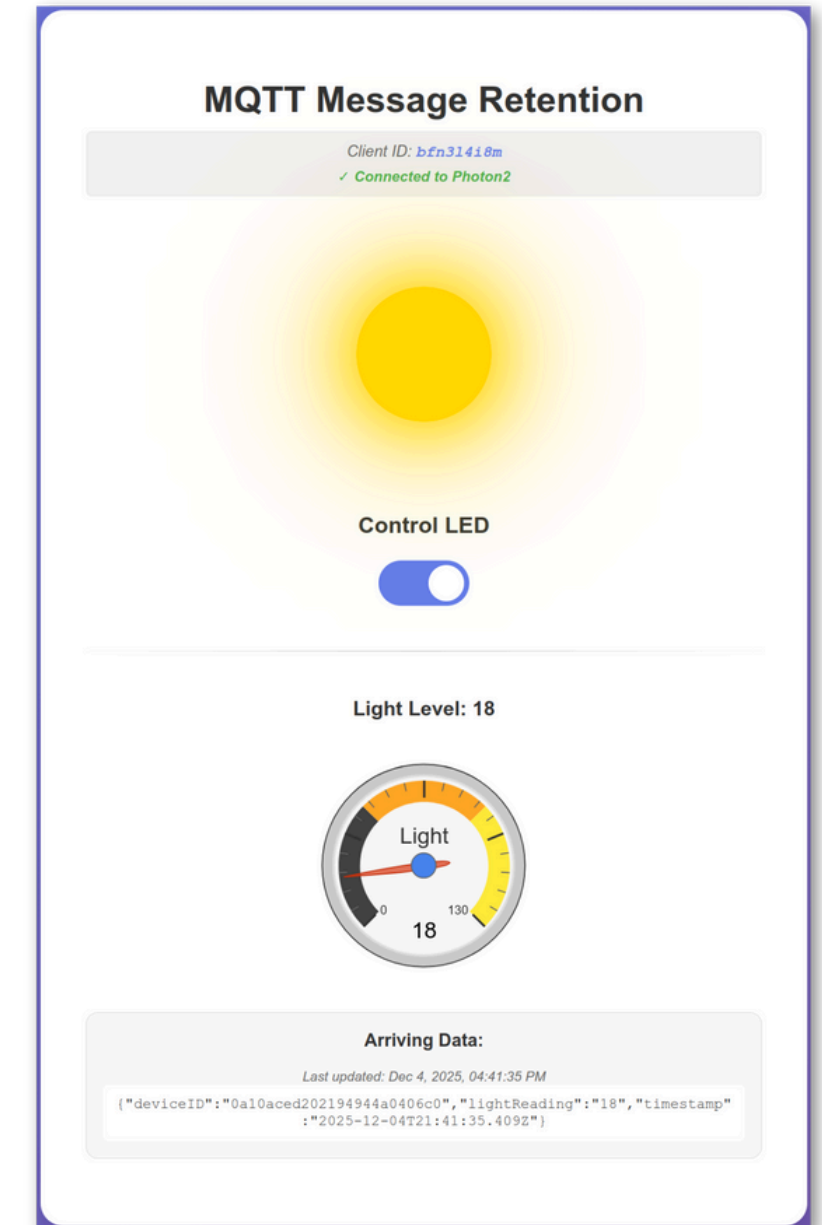


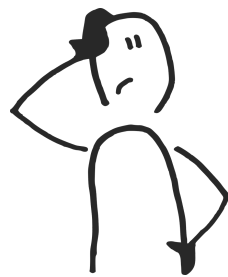
IOT CLASS PROJECT DEMO

Leveraging MQTT Message Retention for Reliable IoT State Management

JOEL ADEBAYO

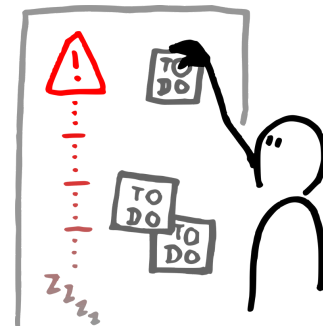


Introduction



Problem

IoT devices often disconnect unexpectedly, causing loss of critical state information. When clients reconnect, they have no knowledge of the system's current state, leading to inconsistent behavior and poor user experience.



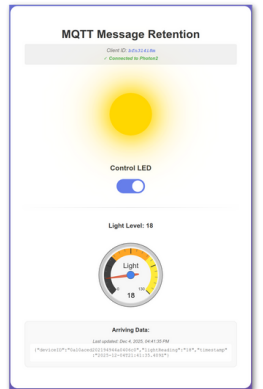
Solution

I implemented MQTT message retention with QoS 1 to ensure the broker stores the last known state. New clients receive retained messages immediately, providing instant synchronization across all devices without manual polling.



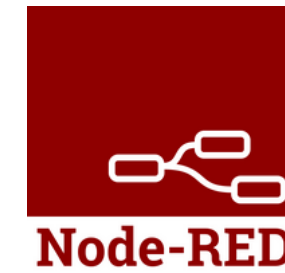
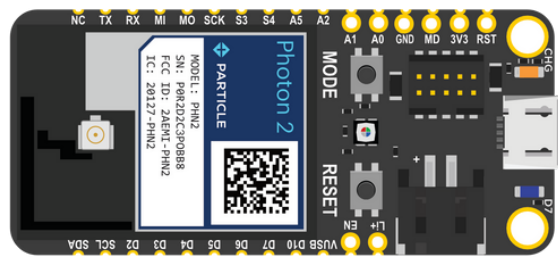
Why It Matters

By ensuring state persistence and instant synchronization, this approach improves system reliability, enhances user experience, reduces network overhead, and enables seamless multi-client coordination in IoT applications.

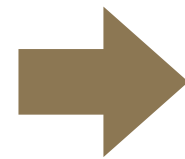


INTRODUCTION

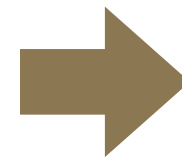
System Architecture 1



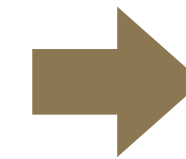
**PHOTON 2
MICRO-
CONTROLLER**



**HTTP POST
(Light Data)**



NODE-RED



**MQTT Broker
(Mosquitto)**

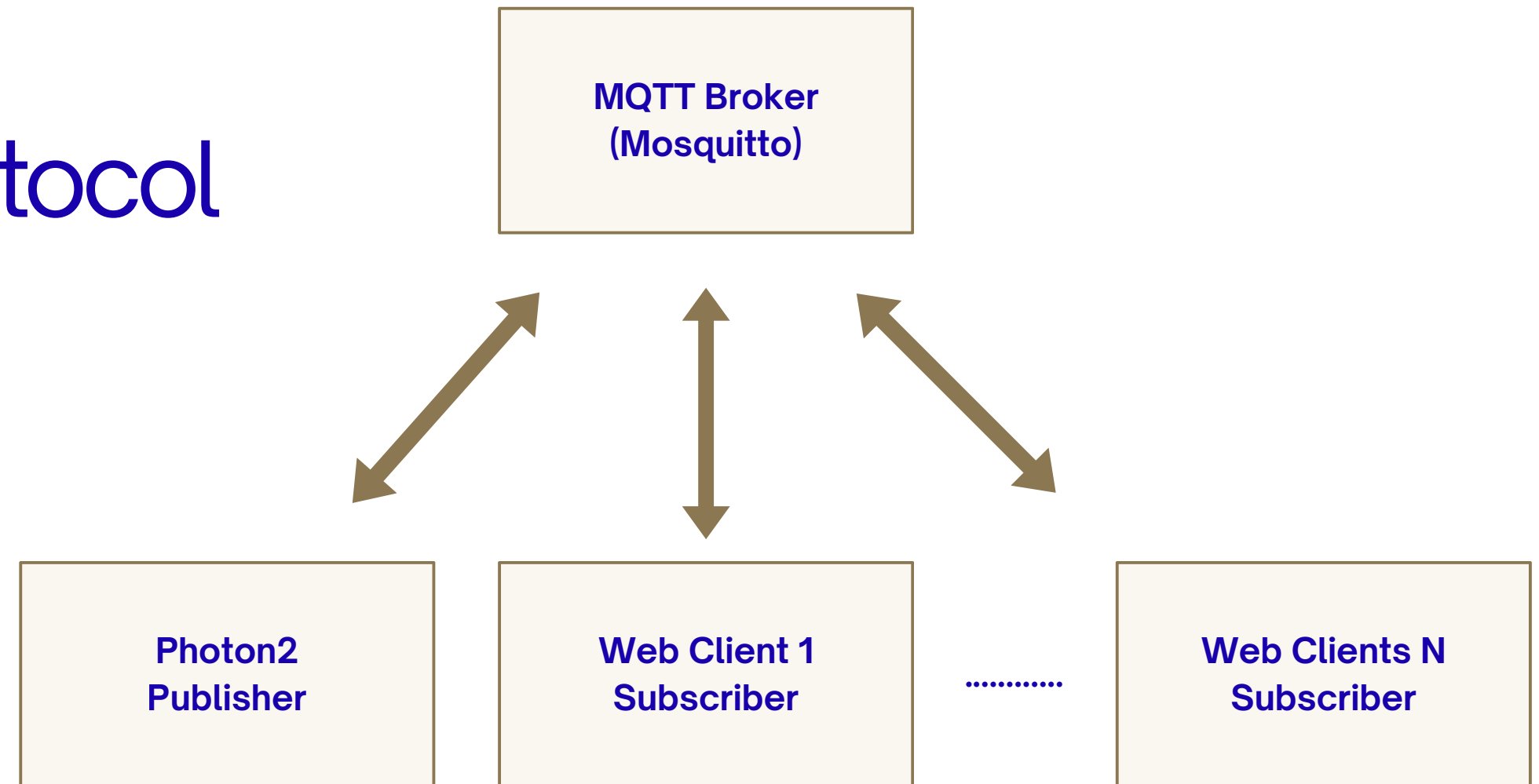
System Architecture 2



MQTT Communication Protocol

Why MQTT?

- **Lightweight:** Minimal bandwidth usage
- **Publish/Subscribe:** Decoupled architecture
- **Real-time:** Instant message delivery
- **Scalable:** Supports many clients
- **Reliable:** QoS levels ensure delivery



MQTT Topics & Message Flow

Topic 1: Photon2LightLevel

Publisher: Node-RED (receives from Photon2 via HTTP)

Subscribers: All web browser clients

```
{ "lightReading": 85, "timestamp": "2025-12-04T14:30:45.123Z" }
```

Topic 2: Photon2LEDControl

Publishers: Web browser clients

Subscriber: Photon2 microcontroller

```
{ "led": "ON", "clientId": "abc123xyz", "timestamp": "2025-12-04T14:30:50.456Z" }
```

Quality of Service (QoS) Levels

QoS Level	Description	Guarantee	Use Case
0	At most once	Fire and forget	Non-critical data
1 ✓	At least once	Acknowledged delivery	This Implementation
2	Exactly once	Four-way handshake	Critical transactions

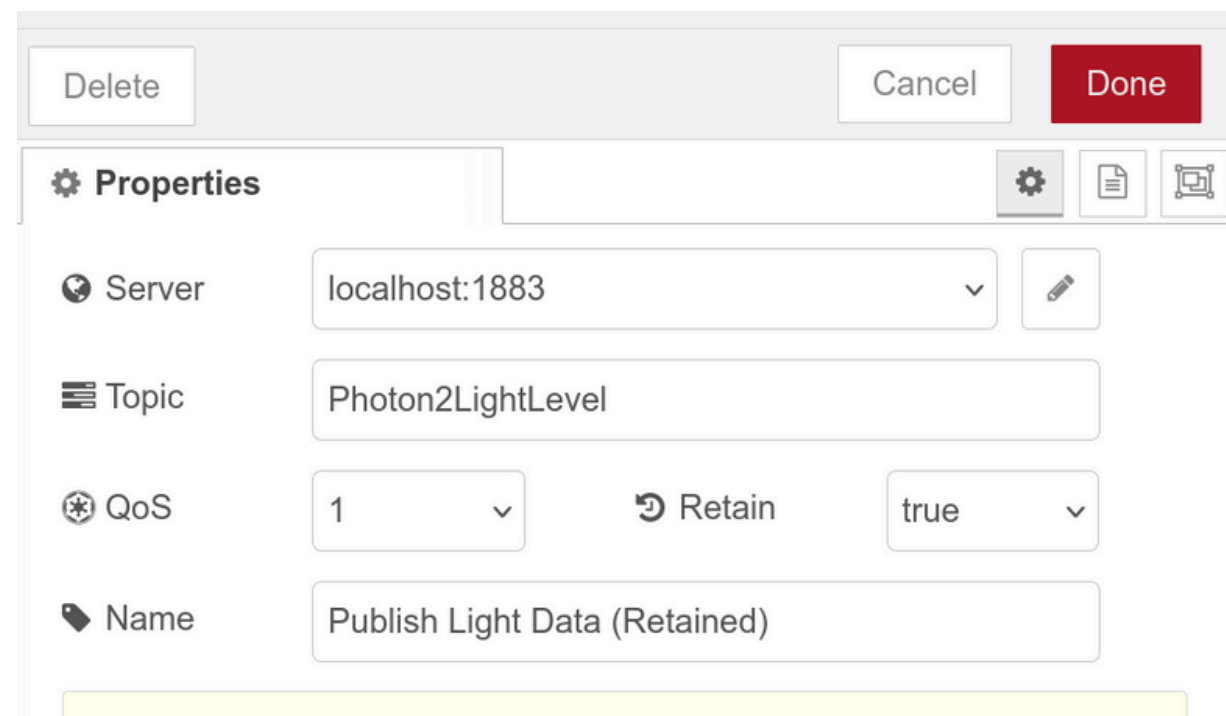
Why QoS 1

Ensures LED commands are delivered reliably
Prevents lost light readings
Balances reliability with performance
Broker acknowledges receipt

MQTT Message Retention

What is Retain?

- Broker stores the last message on a topic
- New subscribers immediately receive it
- Persists until replaced or cleared
- One retained message per topic



A screenshot of the 'Properties' dialog box in MQTT Explorer. The dialog has a title bar with 'Delete', 'Cancel', and 'Done' buttons. The 'Properties' section contains the following fields:

- Server:** localhost:1883
- Topic:** Photon2LightLevel
- QoS:** 1
- Retain:** true
- Name:** Publish Light Data (Retained)

```
1 // Publish LED control message to MQTT with retain flag
2 if (client && client.isConnected()) {
3     const message = new Paho.MQTT.Message(
4         JSON.stringify({
5             led: isOn ? "ON" : "OFF",
6             clientId: clientId,
7             timestamp: new Date().toISOString(),
8         })
9     );
10    message.destinationName = "Photon2LEDControl";
11    message.retained = true; // Enable retain flag
12    message.qos = 1; // QoS level 1 for reliable delivery
13    client.send(message);
14    console.log("Sent LED command:", isOn ? "ON" : "OFF");
15 }
```

Benefits in this scenario

- **Instant Updates:** New clients get current state immediately
- **Multi-Client Sync:** all browsers show consistent data
- **Last Known Value:** Display most recent light reading

Connection States

⚡ Connecting

Initial MQTT broker connection

Action: LED switch disabled

✓ Connected

Receiving fresh data (< 8 seconds old)

Action: LED switch enabled

⚠ Stale

Retained data, but Photon2 offline (> 8s)

Action: Shows "X minutes ago", switch disabled

✗ Disconnected

Complete connection failure

Action: LED switch disabled, retry connection

Detection Parameters:

- **Timeout:** 8 seconds (allows one missed reading)
- **Check Interval:** 500ms (0.5 seconds) for instant detection
- **Photon2 Interval:** 5 seconds between readings

Demo

If you are connected to the CMU Network, you
can scan the QR code to join this demo



Conclusion